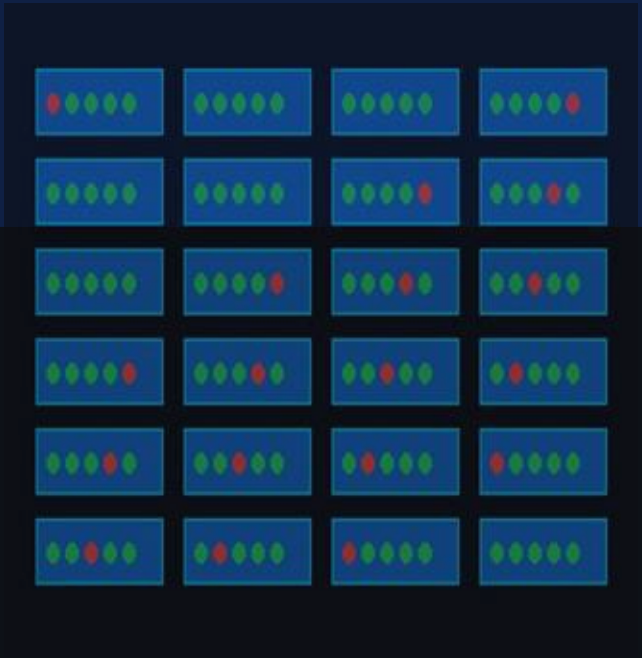
SESSION 9

Anomaly Detection

in IT Operations



Threshold-Based vs ML-Based Anomalies • Multivariate Detection • Kubernetes & Infrastructure Anomalies

Session Agenda

01 **What is Anomaly Detection?**

Definition, types, and importance in IT ops

02 **Threshold-Based Anomalies**

Static & dynamic thresholds, pros and cons

03 **ML-Based Anomaly Detection**

Supervised, unsupervised, and semi-supervised methods

04 **Multivariate Anomaly Detection**

Correlation-aware, PCA, autoencoders

05 **Kubernetes Anomaly Detection**

Pod health, resource anomalies, events

06 **Infrastructure Anomalies & AIOps**

Root cause analysis, observability stack

What is Anomaly Detection?

Anomaly detection is the process of identifying data points, events, or observations that deviate significantly from expected behavior — enabling proactive incident response, fraud prevention, and root cause analysis in IT systems.

Point Anomaly

A single data point deviates significantly from the rest (e.g., CPU spike to 100%)

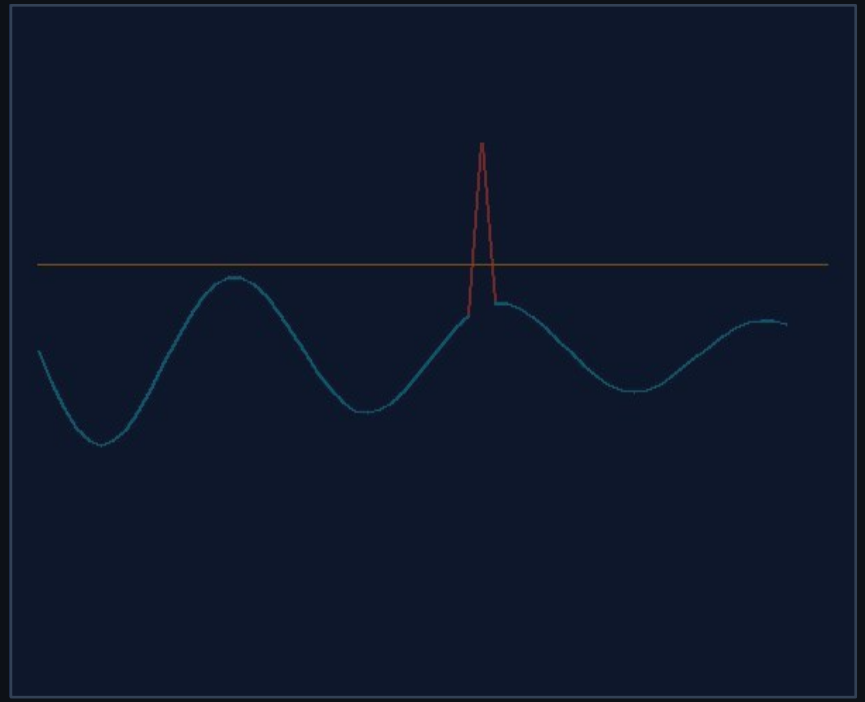
Contextual Anomaly

A value is anomalous given its context (e.g., high load at 3AM vs peak hours)

Collective Anomaly

A group of data points is anomalous together (e.g., distributed DDoS pattern)

Why Anomaly Detection Matters in IT Ops



Mean Time to Detect (MTTD)

Reduced by up to 60% with ML-based detection

False Positive Rate

Threshold-only systems: 40–70% false positive alerts

Incident Response

Automated anomaly triage accelerates RCA by 3–5x

Cost Savings

Early anomaly detection reduces outage costs significantly

Key Use Cases:

- Security & Intrusion Detection
- Infrastructure Health Monitoring
- Performance Degradation Alerts
- Fraud & Compliance

PART 1

Threshold-Based Anomaly Detection

How It Works

- Define a fixed upper/lower bound for a metric (e.g., CPU > 90%)
- Alert fires whenever the observed value crosses that boundary
- Rules can be static (hardcoded) or dynamic (computed from rolling statistics)
- 3-Sigma Rule: flag values > 3 standard deviations from the mean
- Percentile thresholds: alert when metric exceeds 99th percentile
- Hysteresis: requires threshold to be breached for N consecutive samples before alerting

Static Threshold Rule

```
alert if metric > Tupper or metric < Tlower
```

Dynamic (z-score) Rule

```
z = (value – mean) / std_dev
alert if |z| > 3
```

✅ Pros	⚠ Cons
+ Simple, explainable	– No context-awareness
+ Low compute overhead	– High false positive rate
+ Instant deployment	– Manual tuning required
	– Misses correlated events

Static vs Dynamic Thresholds

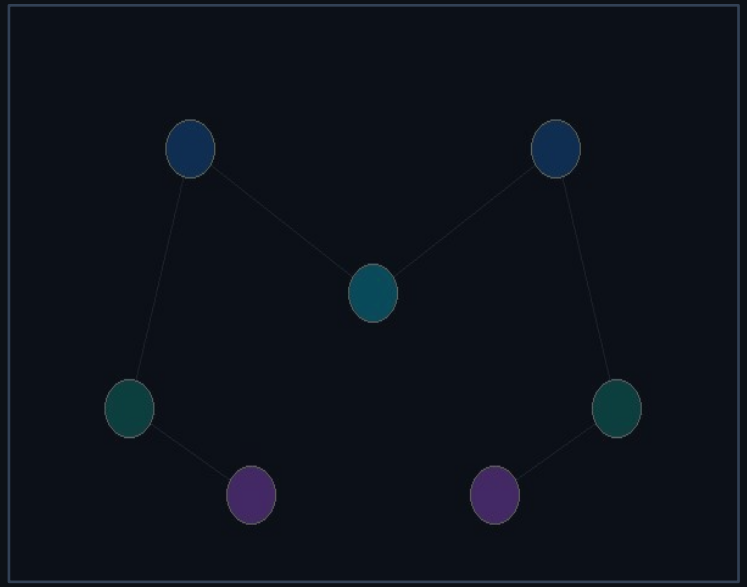
Static Thresholds

- Fixed numeric limit set by an operator
- Example: alert if error_rate > 5%
- Easy to set up — no training data needed
- Does NOT adapt to seasonal patterns or workload changes
- Prone to alert fatigue during peak traffic
- Best for hard limits (SLA caps, safety limits)

Dynamic Thresholds

- Bounds computed from rolling historical data
- Uses mean ± N×σ or percentile-based windows
- Automatically adapts to daily/weekly patterns
- AWS CloudWatch, Datadog, Dynatrace use this approach
- Reduces false positives caused by normal traffic spikes
- Requires sufficient historical data for accuracy

Threshold Limitations & When ML is Needed



When thresholds fail: anomaly lies within bounds but global pattern is abnormal

Concept Drift

Normal behavior changes over time; static thresholds become obsolete without manual updates.

High-Dimensional Data

IT systems have 100s of correlated metrics. Threshold per metric misses cross-metric patterns.

Seasonal Patterns

Traffic at 3 AM vs 3 PM differs wildly. Single threshold causes false alerts during off-peak.

Unknown Anomaly Types

Novel failure modes don't trigger existing rules — zero-day incidents go undetected.

PART 2

ML-Based Anomaly Detection

Supervised

Labeled data: normal vs anomalous. Trains a classifier (Random Forest, SVM, Neural Nets).

✅ Pros

+ High precision when labels available
+ Can detect specific known anomaly classes

⚠ Cons

– Requires labeled dataset (expensive)
– Fails on new/unknown anomaly types

Unsupervised

No labels needed. Learns the normal distribution; anomalies are statistical outliers.

✅ Pros

+ No labeling cost
+ Detects novel anomaly types
+ Works on raw streaming data

⚠ Cons

– Higher false positive rate
– Difficult to tune
– No anomaly class info

Semi-Supervised

Trains on 'normal' data only. Flags deviations from learned normal patterns.

✅ Pros

+ Only normal data needed
+ Good for rare-anomaly domains

⚠ Cons

– Sensitive to data quality
– May learn anomalies as 'normal' if present in training

Key ML Algorithms for Anomaly Detection

Unsupervised

Isolation Forest

Randomly partitions data using trees. Anomalies are isolated in fewer splits. Score 0–1; >0.5 is anomalous.

Semi-Supervised

One-Class SVM

Learns a tight boundary around normal data. Any point outside boundary is anomalous.

Deep Learning

LSTM / RNN

Captures temporal dependencies in time-series. Anomaly = high prediction error vs actual.

Deep Learning

Autoencoder

Learns to compress and reconstruct normal data. High reconstruction error signals an anomaly.

Unsupervised

K-Means Clustering

Anomalies are points far from their nearest cluster center — outside the learned normal zones.

Statistical

ARIMA / Prophet

Forecasts expected time-series values. Observations outside confidence bounds trigger alerts.

Threshold-Based vs ML-Based: Side-by-Side

Dimension	Threshold-Based	ML-Based
Setup Complexity	Low — manual rules	Medium–High — training pipeline
Adaptability	Poor — static rules age out	High — retrains on new data
Explainability	High — clear rule violated	Low–Medium — black box models
Data Requirements	None required	Historical labeled/unlabeled data
Seasonality Support	Limited (dynamic thresh. only)	Native — learns periodic patterns
False Positive Rate	High without tuning	Lower with sufficient training
Operational Cost	Low	Higher (compute, MLOps infra)
Novel Anomalies	Misses unknown types	Can detect unseen anomalies

PART 3

Multivariate Anomaly Detection

Multivariate anomaly detection considers the joint distribution of multiple correlated metrics simultaneously. An anomaly may be invisible in any single metric but clearly visible when correlations between metrics are analyzed together.

🔍 Motivating Example

```
CPU = 60% → Normal
Memory = 55% → Normal
Network I/O = 40% → Normal

↓ BUT all three rising simultaneously = Anomaly!
(Correlated spike = memory leak or cryptominer)
```

PCA

Projects data to principal components; anomalies show large reconstruction error.

Mahalanobis Distance

Measures distance accounting for covariance between features.

LSTM Autoencoder

Sequence model learns temporal multivariate normal — detects correlated deviations.

Graph Neural Networks

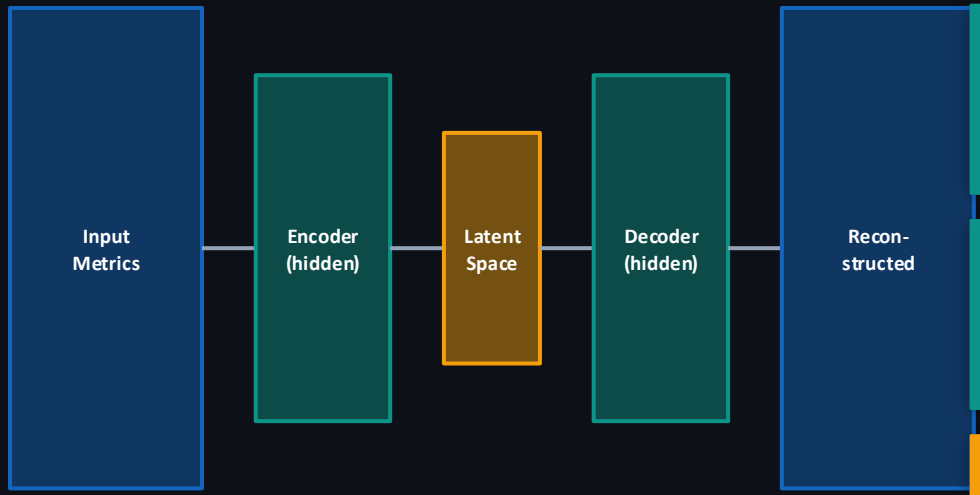
Models dependencies between microservices as a graph; propagates anomaly signals.

PCA & Dimensionality Reduction for Anomaly Detection

Dimensionality reduction maps high-dimensional telemetry data to a low-dimensional subspace where anomalies become visible as outliers

1. **Collect Metrics Matrix**
   Gather N metrics as columns: CPU, mem, disk, latency, error rate...
2. **Standardize Features**
   Z-score normalize each metric to unit variance before PCA.
3. **PCA Decomposition**
   Extract top-k principal components explaining 95%+ of variance.
4. **Compute Reconstruction Error**
   Project data back; large error = cannot be explained by normal patterns.
5. **Threshold on Error Score**
   Flag observations with error > 99th percentile as anomalies.

Autoencoders for Multivariate Anomaly Detection




```
Reconstruction Error = |Input - Reconstructed Output|
```

If error > threshold → ANOMALY DETECTED

Training

Train only on normal operational data — model learns to reconstruct 'healthy' system state perfectly.

Inference

At runtime, compute reconstruction error for each observation; high error = unknown/anomalous state.

LSTM-AE

Combines LSTM (temporal patterns) with autoencoder (compression) — ideal for time-series telemetry.

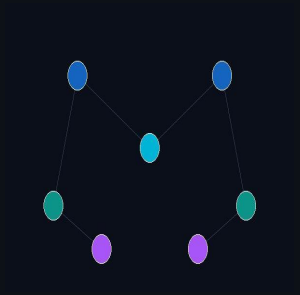
Advantage

No anomaly labels required. Detects both known and novel failure modes from normal-only training.

PART 4

Kubernetes & Infrastructure Anomalies

Why Kubernetes Adds Complexity



- Highly dynamic: pods are ephemeral — metrics appear/disappear constantly
- Multi-layer stack: nodes, pods, containers, services, and control plane all generate telemetry
- Horizontal scaling: autoscaling changes the meaning of 'normal' resource usage
- Service mesh dependencies: cascading failures propagate through interconnected microservices

🔄 Pod Crash Loop

CrashLoopBackOff: pod restarts > N times/min

💥 OOMKilled

Container killed due to memory limit exceeded

⚠️ Node NotReady

Node fails health checks; pods rescheduled

🐌 CPU Throttling

Container throttled > 25% — latency spike risk

⏳ Pending Pods

Pods unschedulable — capacity or resource constraints

🚫 Eviction Events

Kubelet evicts pods on disk/memory pressure

Key Kubernetes Metrics for Anomaly Detection

Node-Level	Pod-Level	Control Plane
node_cpu_usage_seconds_total	container_cpu_usage_seconds_total	apiserver_request_duration_seconds
node_memory_MemAvailable_bytes	container_memory_working_set_bytes	etcd_request_duration_seconds
node_disk_io_time_seconds_total	kube_pod_container_status_restarts_total	scheduler_scheduling_duration_seconds
node_network_receive_bytes_total	kube_pod_status_phase	kube_node_status_condition

Detection Pipeline: Prometheus scrape → PromQL aggregation → ML model scoring → AlertManager → PagerDuty/Slack

Tools: Prometheus, Grafana, Thanos, Datadog, Dynatrace, Amazon CloudWatch Container Insights, OpenTelemetry

Kubernetes Anomaly Detection Architecture

Data Ingestion	Feature Engineering	ML Model Inference	Alert Routing	RCA & Remediation
Prometheus Fluentd OTel Collector	Rolling stats Rate metrics Log parsing	Isolation Forest LSTM-AE AutoML	AlertManager PagerDuty Slack	Runbooks Auto-healing AIOps

Feedback Loop

Confirmed anomalies are fed back as labeled training data → model retrains to improve accuracy over time (online learning). False positives can be marked by SREs to reduce alert fatigue.

Key Tooling:

- Prometheus + Alertmanager
- Grafana Loki (logs)
- Jaeger (traces)
- Datadog APM
- Dynatrace Davis AI
- AWS CloudWatch

Infrastructure-Level Anomalies

Network Anomalies

Packet loss spikes • Unusual port activity • Latency outliers between AZs

Storage / Disk Anomalies

I/O wait >20% • Disk fill rate acceleration • Inode exhaustion

Compute Anomalies

CPU steal time (noisy neighbor) • Thermal throttling events • Unexpected process spawning

Database Anomalies

Query latency p99 spike • Connection pool saturation • Replication lag growth

AIOps & Automated Root Cause Analysis

1

Anomaly Detected

ML model scores a metric as anomalous; alert fires

2

Event Correlation

AIOps correlates concurrent anomalies across services

3

Causal Graph Analysis

Service dependency graph used to trace upstream cause

4

Pattern Matching

Compare with historical incident signatures/runbooks

5

Remediation Trigger

Auto-heal (restart pod, scale HPA) or alert SRE team

AIOps Platforms

Dynatrace Davis

Full-stack AI causation engine

Datadog WatchDog

Automatic anomaly detection + root cause

IBM AIOps Insights

Event correlation + alert noise reduction

Moogsoft

Alert clustering + incident correlation

PagerDuty AIOps

Intelligent alert grouping and ML triage

BigPanda

Event correlation with topology awareness

Observability Stack: The Three Pillars

Metrics

Numerical time-series data (CPU, latency, error_rate). Efficient to store and query. Suitable for threshold + ML anomaly detection.

Key Tools:

- Prometheus
- InfluxDB
- Datadog Metrics
- CloudWatch

Logs

Unstructured/semi-structured event records from apps and infra. Rich context but high volume. ML used for log anomaly scoring.

Key Tools:

- Elasticsearch
- Loki
- Splunk
- Fluentd

Traces

Distributed request traces across microservices. Critical for latency anomalies and cascading failure detection across service graphs.

Key Tools:

- Jaeger
- Zipkin
- AWS X-Ray
- Tempo

Best Practices in IT Anomaly Detection

01 Start with Thresholds, Then Augment with ML

Use rule-based detection for known hard limits. Add ML layers for pattern learning and correlation.

02 Reduce Alert Fatigue with Deduplication

Cluster related alerts. AIOps platforms group similar events into a single incident for triage.

03 Use Multivariate Models for Microservices

Single-metric detection misses correlated failures. Model the joint distribution of dependent metrics.

04 Implement Feedback Loops for Retraining

SRE-confirmed incidents become training labels. Online learning updates the model continuously.

05 Monitor Data Quality & Drift

Data pipeline failures and schema changes invalidate anomaly models. Alert on missing/noisy data.

06 Align Anomaly Scores with Business SLOs

Not all anomalies matter equally. Weight scoring by user impact, error budget consumption, and revenue risk.

Tools & Platforms for Anomaly Detection

Open Source

- Prometheus + AlertManager
- Grafana + ML plugins
- Netdata (K-Means built-in)
- PyOD — Python anomaly library
- scikit-learn Isolation Forest
- Apache Flink (stream processing)

Commercial / SaaS

- Datadog WatchDog
- Dynatrace Davis AI Engine
- New Relic AI
- Splunk ITSI
- Elastic Observability
- Sumo Logic Machine Learning

Cloud-Native

- AWS CloudWatch Anomaly Detection
- Azure Monitor + ML
- GCP Cloud Operations
- Amazon DevOps Guru
- Azure Defender for Cloud
- GCP Error Reporting AI

Challenges in Real-World Anomaly Detection

Label Scarcity

Real anomalies are rare — getting enough labeled examples for supervised learning is expensive and slow.

Alert Fatigue

Over-sensitive models fire too many false alarms. SRE teams start ignoring alerts — defeating the purpose.

High Cardinality Metrics

Modern microservices emit millions of time-series. Scaling ML inference across all metrics is computationally hard.

Concept Drift

System behavior evolves with new deployments, traffic growth, and seasonal changes — models become stale.

Interpretability

ML models like LSTMs and autoencoders are black boxes. Engineers need explainable alerts for effective RCA.

Multi-Tenant Environments

Shared infrastructure makes it hard to distinguish tenant-specific anomalies from platform-wide failures.

Key Takeaways

Threshold-based detection is simple and explainable but fails on complex, seasonal, and correlated patterns in modern IT systems.

ML-based methods (Isolation Forest, LSTM, Autoencoders) adapt to changing baselines and detect unknown failure modes automatically.

Multivariate anomaly detection is essential for microservices: individual metrics may be normal while joint behavior signals a failure.

Kubernetes anomaly detection requires monitoring ephemeral pods, node conditions, and control plane health with auto-adapting models.

A feedback loop (alert → confirm → label → retrain) continuously improves model accuracy and reduces false positive rates.

Combine the three observability pillars — metrics, logs, and traces — with AIOps for holistic, actionable anomaly detection.